

THE "ARCHITECTURAL STRESS TEST"

To confirm you possess the technical depth to build our SaaS, you are required to submit the following **Technical Design Document (TDD)** for our Leads system. I do not want generic tutorials; I want your specific logic for our architecture.

Please provide your responses in a clear document.

Section 1: Database Multi-Tenancy (The Security Proof)

We have a table agents and a table leads. A lead belongs to one agent.

Provide the SQL script for:

1. The RLS policy that ensures an agent can **only** read their own leads.
2. The RLS policy that ensures an agent can **only** update a lead if that lead belongs to them (using a JOIN or EXISTS clause).
3. Explain, in your own words, the difference between USING and WITH CHECK clauses in Supabase RLS. Why is this critical for data integrity during updates?

Section 2: Resilience & Error Handling (The Scalability Proof)

We are processing a batch of 500 leads through an external AI API.

1. If the API returns a 429 Rate Limit or 503 Service Unavailable error, explain exactly how you will design the retry mechanism.
2. Provide a pseudo-code or diagram showing where the 'state' of the batch is stored during this retry process (e.g., in the database, in a queue, or in local state?).

Section 3: The "Worst-Case" Analysis (The Grit Proof)

Describe a specific, real-world production bug you personally caused that took down a service.

1. What was the root cause?
2. How did you detect it (logs, monitoring, user complaints)?
3. What **automated test** or **monitoring logic** did you implement immediately after to ensure that specific failure mode could *never* happen again?

Submission Requirement:

I expect a detailed, technical response for each section. If you use external documentation, please cite the specific links you are referencing. Your ability to provide concrete, system-level logic will determine your fitness for this contract.